SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3 **Total Marks:30**

Annexure A

Dry Run Evaluation

Code of Conduct:

I _Theerthi (192473026) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

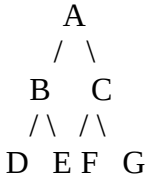
B Theerthi(192473026)
Signature of the student:

nnexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph:



Task: Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

Iteration	Queue	Visited Node
1		
2		
3		
4		
5		
6		
7		

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

Code

```
from collections import deque

graph = {

    'A': ['B', 'C'],

    'B': ['D', 'E'],

    'C': ['F', 'G'],

    'D': [],

    'E': [],

    'F': [],

    'G': []
```

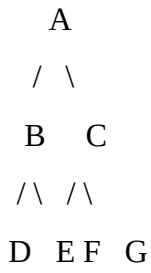
```
}
def bfs (graph, start):
visited = []
queue = deque([start])
 print ("\breadth-FIRST SEARCH (BFS)")
 print ("-" * 40)
iteration = 1
while queue:
print (iteration {iteration}")
 print ("Queue Before :", list(queue))
  node = queue. pop left ()
 if node not in visited:
 visited. Append(node)
 for neighbor in graph[node]:
  if neighbor not in visited:
  queue. Append(neighbor)
 print ("Visited Node :", node)
  print ("Queue After  :", list(queue))
   print ()
  iteration += 1
 print ("BFS Traversal Order:", " -> ".join(visited))
```

Output BFS

```
BREADTH-FIRST SEARCH (BFS)
----------------------------------------
Iteration 1
Queue Before : ['A']
Visited Node : A
Queue After  : ['B', 'C']

Iteration 2
Queue Before : ['B', 'C']
Visited Node : B
Queue After  : ['C', 'D', 'E']

Iteration 3
Queue Before : ['C', 'D', 'E']
Visited Node : C
Queue After  : ['D', 'E', 'F', 'G']

Iteration 4
Queue Before : ['D', 'E', 'F', 'G']
Visited Node : D
Queue After  : ['E', 'F', 'G']

Iteration 5
Queue Before : ['E', 'F', 'G']
Visited Node : E
Queue After  : ['F', 'G']

Iteration 6
Queue Before : ['F', 'G']
Visited Node : F
Queue After  : ['G']

Iteration 7
Queue Before : ['G']
Visited Node : G
Queue After  : []

BFS Traversal Order: A -> B -> C -> D -> E -> F -> G
```

2. Depth-First Search (DFS) Using the same graph,

Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		



Perform **Depth-First Search (DFS)** starting from **A**.

Code

```
def dfs(graph, start):
 visited = []
 stack = [start]
 print ("\nDEPTH-FIRST SEARCH (DFS)")
  print ("-" * 40)
 iteration = 1
 while stack:
 print(iteration {iteration}")
  print ("Stack Before:", stack)
  node = stack. Pop()
  if node not in visited:
  visited. Append(node)
  for neighbor in reversed(graph[node]):
  if neighbor not in visited:
  stack. Append(neighbor)
  print ("Visited Node:", node)
  print ("Stack After :", stack)
  print()
  iteration += 1
 print ("DFS Traversal Order:", " -> ".join(visited))
bfs(graph, 'A')
dfs(graph,'A')
```

Output DFS

```
DEPTH-FIRST SEARCH (DFS)
----------------------------------------
Iteration 1
Stack Before: ['A']
Visited Node : A
Stack After  : ['C', 'B']

Iteration 2
Stack Before: ['C', 'B']
Visited Node : B
Stack After  : ['C', 'E', 'D']

Iteration 3
Stack Before: ['C', 'E', 'D']
Visited Node : D
Stack After  : ['C', 'E']

Iteration 4
Stack Before: ['C', 'E']
Visited Node : E
Stack After  : ['C']

Iteration 5
Stack Before: ['C']
Visited Node : C
Stack After  : ['G', 'F']

Iteration 6
Stack Before: ['G', 'F']
Visited Node : F
Stack After  : ['G']

Iteration 7
Stack Before: ['G']
Visited Node : G
Stack After  : []

DFS Traversal Order: A -> B -> D -> E -> C -> F -> G
```

3. **Initial State :**

1	**3**	**6**
5	**0**	**2**
4	**7**	**8**

Goal State :

1	**2**	**3**
4	**5**	**6**
7	**8**	**0**

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.
Fill in the table.

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1			
2			
3			
4			
5			

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

Code

```
from collections import deque

initial = (

    (1, 3, 6),

    (5, 0, 2),

    (4, 7, 8)

)

goal = (

    (1, 2, 3),

    (4, 5, 6),

    (7, 8, 0)

)

def find_blank(state):

for i in range(3):

for j in range(3):

 if state[i][j] == 0:

 return i, j

def get_neighbors(state):

 row, col = find_blank(state)
```

```
    moves = [(-1,0), (1,0), (0,-1), (0,1)]  # Up, Down, Left, Right
    neighbors = []
  for dr, dc in moves:
   nr = row + dr
   nc = col + dc
  if 0 <= nr < 3 and 0 <= nc < 3:
  temp = [list(r) for r in state]
temp[row][col], temp[nr][nc] = temp[nr][nc], temp[row][col]
neighbors.append(tuple(tuple(r) for r in temp
 return neighbors
def print_state(state):
    for row in state:
    print(row)
    print()
def bfs(initial, goal):
queue = deque([(initial, [initial])])
visited = set( generated = 1
iteration = 1
 while queue:
print("="*50)
 print("Iteration:", iteration)
print("\nQueue Before Expansion:", len(queue))
```

```
current, path = queue.popleft()
print("\nCurrent State:")
 print_state(current)
if current == goal
print("GOAL STATE REACHED!\n")
print("Solution Path:\n")
for step in path:
 print_state(step)
print("Total Moves:", len(path)-1)
print("Total Generated States:", generated)
return
 visited.add(current)
for neighbor in get_neighbors(current):
if neighbor not in visited and all(neighbor != s for s,_ in queue):
 queue.append((neighbor, path+[neighbor]))
 generated += 1
print("Queue After Expansion:", len(queue))
 iteration += 1
bfs(initial, goal)
```

Output

```
(1, 3, 6)
(5, 0, 2)
(4, 7, 8)

(1, 3, 6)
(5, 2, 0)
(4, 7, 8)

(1, 3, 0)
(5, 2, 6)
(4, 7, 8)

(1, 0, 3)
(5, 2, 6)
(4, 7, 8)

(1, 2, 3)
(5, 0, 6)
(4, 7, 8)

(1, 2, 3)
(0, 5, 6)
(4, 7, 8)

(1, 2, 3)
(4, 5, 6)
(0, 7, 8)

(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

(1, 2, 3)
(4, 5, 6)
(7, 8, 0)

Total Moves: 8
```